



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра математического обеспечения и стандартизации
информационных технологий

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2

по дисциплине «Тестирование и верификация
программного обеспечения»

**Тема: «Модульное и мутационное тестирование
программного продукта»**

Команда «Дерби» · Дата выполнения: 09.10.2026

Состав: Дубейко А. С., Домбровский Я. А.

Выполнил:
Студент группы
ИКБО-61-24

Дубейко Аким Сергеевич

Проверил:

Доцент кафедры МОСИТ,
кандидат технических наук, доцент
Чернов Е. А.

Москва 2026

1 Цель, задачи и организация работы

Цель — освоить модульное и мутационное тестирование на обновлённой игре «Дерби у стадиона»: разработать проверяемые контракты функций, выявить преднамеренный дефект, проверить исправление, измерить покрытие и улучшить набор тестов по результатам мутаций.

В новой версии доступны десять испанских клубов и независимый выбор команды бросающих болельщиков и команды персонажа в центре. При запуске бросают болельщики «Барселоны», в центре находится болельщик «Реал Мадрид».

Таблица 1 — Состав команды и распределение ролей

Участник, группа ИКБО-61-24	Разработка	Взаимное тестирование
Дубейко Аким Сергеевич	model.py	game.py
Домбровский Ярослав Андреевич	game.py	model.py

Индивидуальный отчёт посвящён модулю model.py. Разработчик по принятому распределению — Дубейко Аким Сергеевич, тестировщик — Домбровский Ярослав Андреевич. Каждый участник одновременно рассматривается как тестировщик модуля партнёра.

Задачи: описать не менее пяти функций; подготовить базовый набор тестов; внести одну учебную ошибку в модуль; зарегистрировать дефект и проверить исправление; создать мутанты; исследовать выжившие варианты; дополнить тесты и повторить измерения.

Все результаты ниже получены автоматическими запусками. Учебные ошибки внесены в изолированные копии кода. Рабочая версия приложения оставлена исправной. Распределение ролей не подменяет журналы фактических запусков.

2 Обновлённое приложение

Пробел запускает бросок, клавиши 1–3 выбирают предмет. Меню открывается клавишей Т или кнопкой «Команды»; Enter, Esc и кнопка «Играть» возвращают в игру. В меню время симуляции остановлено. Выбор другого клуба очищает текущий раунд. Выбор клуба соперника меняет стороны местами. R очищает раунд, сохраняя команды. R очищает раунд, сохраняя команды.

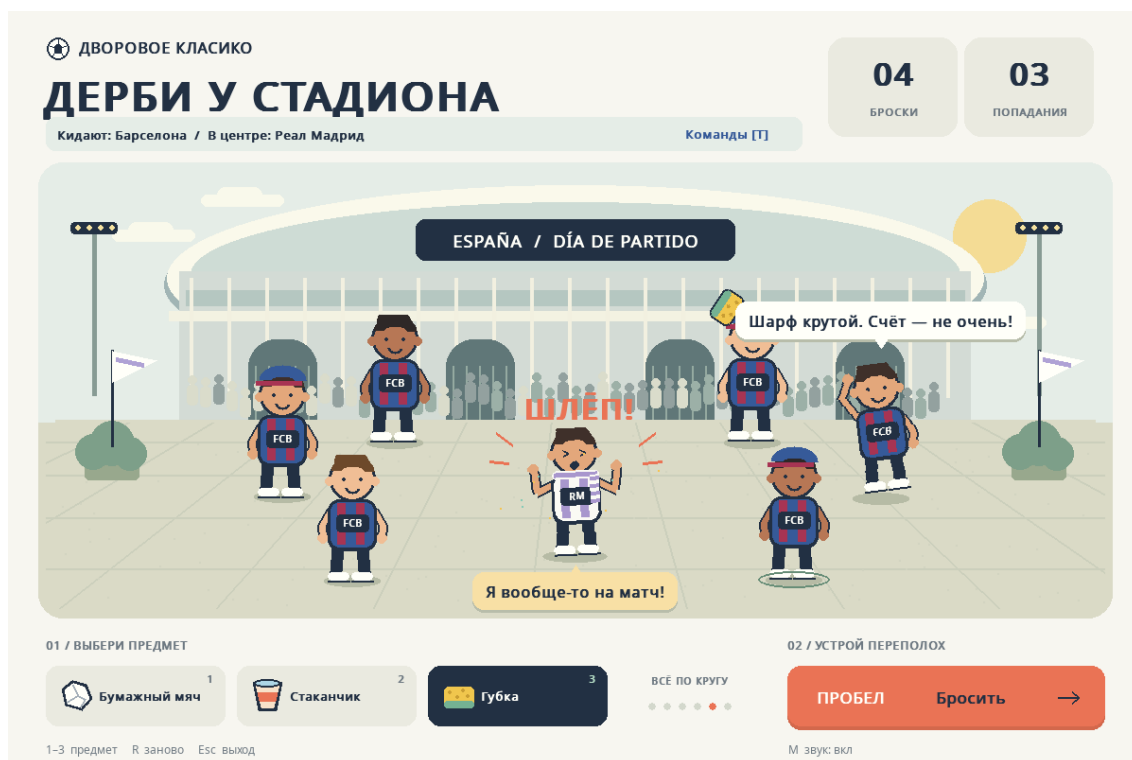


Рисунок 1 — Новая версия: «Барселона» бросает, «Реал» в центре

Доступные клубы: «Барселона», «Реал Мадрид», «Атлетико Мадрид», «Севилья», «Реал Бетис», «Валенсия», «Вильярреал», «Атлетик Бильбао», «Реал Сосьедад», «Жирона». Цвета формы, шарфы, обозначения и реплики зависят от выбранных сторон.

2.1 Меню и среда эксперимента



Рисунок 2 — Выбор двух команд

Таблица 2 — Воспроизводимость

Параметр	Значение
Версия приложения	Коммит 28a524c; исходники в src/
Стенд	Linux, Python 3.12.14
Библиотеки	pygame-ce 2.5.8; pytest 9.1.1; coverage.py 7.16.2
Тестовый подход	unittest, pytest, Mock; SDL dummy для интерфейса
Мутационный инструмент	run_experiments.py: 28 контролируемых замен
Материалы	experiments_practice2_teams.zip; summary.json, JUnit, HTML coverage

3 Функции и контракты модуля

Таблица 3 — Функциональность model.py

Функция	Контракт
Projectile.progress	Ограничивает прогресс полёта диапазоном [0; 1].
Projectile.position	Вычисляет координаты снаряда на дуге.
GameState.reset	Очищает раунд, сохраняя выбранные команды.
GameState.choose_club	Проверяет роль и индекс; меняет клуб; при совпадении меняет стороны; очищает раунд.
GameState.select	Выбирает допустимый индекс предмета 0–2.
GameState.throw	Создаёт бросок при истечении задержки 0,25 с и формирует реплику.
GameState.update	Проверяет dt и обновляет движение, попадания и частицы.

Ключевые ожидаемые значения: исходные индексы сторон (0; 1); после choose_club("target", 0) — (1; 0). После смены команды счётчики и снаряды очищены. При открытом меню update не вызывается. Для окна 1600 × 800 область сцены — 1200 × 800, смещение — (200; 0).

Для полёта $t = \min(1, \max(0, \text{age} / \text{duration}))$, $x = x_0 + (x_1 - x_0)t$, $y = y_0 + (y_1 - y_0)t - 580t(1 - t)$. В середине полёта из (100; 400) в (600; 500) ожидается (350; 305).

4 Модульное тестирование и улучшение набора

Базовые тесты проверяют обычные сценарии, независимость модели от графики, обработку событий через Mock и два новых контракта: обмен сторон и остановку времени в меню. Улучшенный набор добавляет границы, отрицательные сценарии и проверку реальных кликов в безэкранным SDL.

Таблица 4 — Фактические результаты корректных версий

Набор	Основные случаи	Ошибки / пропуски
baseline	22	0 failures; 0 errors; 0 skipped
improved	48	0 failures; 0 errors; 0 skipped
integration	48	0 failures; 0 errors; 0 skipped

Это общие наборы обоих модулей. Базовый набор содержит 22 основных теста, улучшенный — 48. Внутри unittest дополнительно выполняются 11 subtests; JUnit поэтому сообщает 33 и 59 проверок. Основные тестовые случаи и внутренние подтесты не смешиваются.

В улучшенном наборе четыре сценария из test_teams.py включают один интерфейсный сценарий с настоящим объектом Game и безэкраным SDL. Поэтому результаты полного набора включают модульные проверки и проверку взаимодействия компонентов, а не только изолированные unit-тесты.

Таблица 5 — Добавленные сценарии новой версии

Сценарий	Ожидание
Начальные команды и R	Барселона → Реал; R сохраняет выбранные клубы
Выбор клуба соперника	Команды меняются местами; текущий бросок очищен
Все клубы на обеих сторонах	Правильный индекс, разные стороны, корректная реплика
Клики после изменения размера	Меню открывается; каждый клуб выбирается
Пауза и Esc в меню	Нет update и броска; Esc закрывает меню, сохраняя игру

5 Преднамеренная ошибка и её исправление

Таблица 6 — Карточка дефекта

Поле	Описание
Идентификатор	BUG-MODEL-02
Модуль / функция	model.py / GameState.choose_club
Разработчик	Дубейко Аким Сергеевич
Тестировщик	Домбровский Ярослав Андреевич
Категория	Major
Статус	Open → Fixed → Closed
Шаги	Создать GameState; вызвать choose_club("target", 0); прочитать throwers и target.
Ожидание	(1; 0): Реал бросает, Барселона в центре.
Фактически в дефектной копии	(0; 0): обе стороны стали Барселоной.
До исправления	1 failed; 21 passed
После исправления	22 passed; 0 failures; 0 errors

Намеренно ошибочный фрагмент:

```
pass
```

Исправленный фрагмент:

```
setattr(self, other, getattr(self, role))
```

Исправление проверено повторным запуском базового набора на исходной корректной копии. Файлы дефектных вариантов сохранены в results/BUG-MODEL-02-open/.

5.1 Журнал обнаружения и взаимная проверка

```
.....F. [100%]
===== FAILURES =====
_____ test_model_swap_opponent _____
tests/test_team_contracts.py:8: in test_model_swap_opponent
    assert (state.throwers, state.target) == (1, 0)
E   assert (0, 0) == (1, 0)
E
E   At index 0 diff: 0 != 1
E   Use -v to get more diff
===== short test summary info =====
FAILED tests/test_team_contracts.py::test_model_swap_opponent - assert (0, 0)...
1 failed, 21 passed, 11 subtests passed in 0.10s
```

Повторная проверка исправленного варианта:

```
..... [100%]
22 passed, 11 subtests passed in 0.09s
```

Взаимное тестирование модуля партнёра: BUG-GAME-02 — Открытое меню не приостанавливает игру. Базовый набор обнаружил 1 ошибку утверждения; после исправления все 22 основных теста прошли. Полные журналы обоих дефектов приложены в архиве.

6 Анализ покрытия кода

Таблица 7 — Покрытие model.py

Набор	Исполняемые строки	Ветви
baseline	119/121 (98,35%)	18/22 (81,82%)
improved	121/121 (100,00%)	22/22 (100,00%)
integration	121/121 (100,00%)	22/22 (100,00%)

baseline — базовые проверки; improved — усиленный набор, включая интерфейсный сценарий; integration — тот же набор вместе со smoke-запуском игры. Проценты строк и ветвей рассчитаны отдельно по coverage.json, а не взяты из объединённой оценки coverage.py.

Покрытие показывает выполнение кода, но не доказывает точность утверждений. Мутант может выжить в уже выполненной строке, если тест проверяет только направление движения вместо точных координат или не проверяет отсутствие лишнего звука.

Журнал общего покрытия после безэкранного запуска:

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
game.py	462	22	180	14	94%	112-113, 116->exit, 120->exit, 410-411, 430, 455-457, 469, 473, 477->482, 487, 491, 494, 497, 507, 511-513, 519-522
model.py	121	0	22	0	100%	
TOTAL	583	22	202	14	95%	

Непокрытые строки game.py относятся в том числе к обычному непрерывному циклу приложения и ветвям ошибок запуска. SDL dummy не проверяет физическое аудиоустройство, качество звучания и поведение оконного менеджера macOS. Эти ограничения не скрываются общей оценкой покрытия.

7 Методика мутационного тестирования

Использован собственный ограниченный генератор `run_experiments.py`. Он автоматически применяет одну замену, проверяет единственность исходного фрагмента и синтаксис через `ast.parse`. Каждый вариант запускается в новой временной папке отдельным процессом `pytest` с тайм-аутом 40 секунд. Изменённый исходник, описание мутации, `stdout` и `JUnit` сохраняются.

Мутант считается обнаруженным только при реальном провале утверждения теста. Ошибки импорта, сбора тестов и тайм-ауты учитываются отдельно. Перед мутациями обязательно проходит корректный код. После улучшения используется та же выборка; это позволяет сравнить результаты.

Операторы: AOR — изменение арифметики; ROR — изменение отношения; LOR — изменение логики; CRP — замена значения; SDL — удаление действия или части условия. Всего проверено 28 мутантов: 18 для `model.py` и 10 для `game.py`.

Индивидуальная выборка `model.py`: обнаружено 11/18 (61.11%) до улучшения и 18/18 (100.00%) после. Формула: $\text{Mutation score} = \frac{\text{обнаруженные}}{\text{валидные мутанты}} \times 100\%$.

Все мутанты запускались на общем наборе из 22 и 48 основных тестов; индивидуальные показатели являются срезом по модулю. Учебные дефекты рассматриваются отдельно и не добавляются к знаменателю 28 повторно.

7.1 Результаты по мутантам model.py

Таблица 8 — Обнаруженные и выжившие варианты

ID	Изменение	До	После
M01	Дуга направлена вниз	Обнаружен	Обнаружен
M02	Высота дуги уменьшена	Выжил	Обнаружен
M03	Снят верхний предел прогресса	Обнаружен	Обнаружен
M04	Снят нижний предел прогресса	Выжил	Обнаружен
M05	Принят недопустимый индекс предмета	Выжил	Обнаружен
M06	Инвертирован запрет броска	Обнаружен	Обнаружен
M07	Бросок разрешён до конца задержки	Выжил	Обнаружен
M08	Последний фанат пропущен	Обнаружен	Обнаружен
M09	Броски считаются дважды	Обнаружен	Обнаружен
M10	Попадание не фиксируется на границе полёта	Обнаружен	Обнаружен
M11	Попадание не увеличивает счётчик	Обнаружен	Обнаружен
M12	Ослаблена проверка времени	Обнаружен	Обнаружен
M13	Нулевой шаг ошибочно запрещён	Обнаружен	Обнаружен
M14	Отключено ускорение частиц	Выжил	Обнаружен
M15	Частица сохраняется при нулевом времени жизни	Выжил	Обнаружен
M16	Горизонтальный полёт в обратную сторону	Обнаружен	Обнаружен
M17	Не меняется сторона соперника	Обнаружен	Обнаружен
M18	Смена клуба не очищает раунд	Выжил	Обнаружен

7.2 Анализ выживших вариантов

Таблица 9 — Причины и доработка

ID	Недостаток базового набора	Добавленная проверка
M02	Нет точной высоты дуги	Координаты в середине полёта
M04	Нет отрицательного age	Нижняя граница progress
M05	Нет недопустимого индекса	Индексы -1, 3, 100
M07	Нет проверки перед границей задержки	Бросок за 0,001 с до окончания
M14	Не проверяется ускорение	Точное изменение скорости частицы
M15	Нет границы life=0	Удаление ровно при нулевой жизни
M18	Не проверялся сброс раунда при смене клуба	Выбор клуба во время полёта

Для новых функций базовый набор уже обнаруживал отсутствие обмена сторон (M17) и паузы (G09). Дополнительные тесты закрыли сброс раунда (M18) и открытие меню мышью (G10). Все исходные журналы доступны для проверки каждого результата.

8 Выводы

Модуль model.py описан через явные контракты не менее пяти функций. Преднамеренный дефект BUG-MODEL-02 воспроизведён, обнаружен тестом и устранён; повторная проверка корректной версии успешна. Учебные изменения не внесены в пользовательскую игру.

Для индивидуальной выборки качество обнаружения повысилось с 11/18 до 18/18. Усиление обеспечили точные утверждения о численных результатах, границах, состоянии обеих сторон и отсутствии лишних действий.

Общий результат: 22 базовых и 48 улучшенных тестов проходят. Обнаружено 15/28 мутаций до улучшения и 28/28 после. Ошибки сбора, таймауты и невалидные варианты не выявлены.

100% относится только к выбранным 28 изменениям и не означает отсутствия любых ошибок. Эквивалентные мутанты в данной выборке не установлены. Следующие шаги — расширение операторов, длительные последовательности выбора команд и ручная проверка звука и окон на macOS.

Список использованных источников

1. Практическая работа № 2. Модульное и мутационное тестирование программного продукта. Предоставленная методичка.

2. ГОСТ 19.401-78. ЕСПД. Текст программы. Требования к содержанию и оформлению.

3. ГОСТ 34.602-2020. Техническое задание на создание автоматизированной системы.

4. pytest documentation. <https://docs.pytest.org/>

5. coverage.py documentation. <https://coverage.readthedocs.io/>

6. pygame-ce documentation. <https://pyga.me/docs/>

7. Исходники игры: <https://github.com/akimdubeyko-dotcom/calculator> (коммит 28a524c).

Дата обращения к электронным ресурсам: 09.10.2026.

Приложение А. Исследуемый исходный код

Полные исходники обоих модулей находятся в src/ архива экспериментов. Ниже приведены функции индивидуального модуля. Код соответствует обновлённой версии.

```
class Projectile:
    start: tuple[float, float]
    end: tuple[float, float]
    kind: int
    duration: float = 0.8
    age: float = 0.0

    @property
    def progress(self):
        return min(1.0, max(0.0, self.age / self.duration))

    @property
    def position(self):
        t = self.progress
        return (
            self.start[0] + (self.end[0] - self.start[0]) * t,
            self.start[1] + (self.end[1] - self.start[1]) * t - 145 * 4 * t * (1 - t),
        )
```

Приложение А — reset

```
def reset(self):
    self.time = 0.0
    self.selected = 0
    selfthrows = 0
    self.hits = 0
    self.cooldown = 0.0
    self.reaction = 0.0
    self.projectiles = []
    self.particles = []
    self.poses = [0.0] * len(FANS)
    self.speaker = 0
    self.bubble = ""
    self.bubble_time = 0.0
    self.reply = ""
    self.reply_time = 0.0
    self.last_kind = 0
```

Приложение А — choose_club

```
def choose_club(self, role, index):
    """Change a side, swapping opponents if necessary, and start a fresh round."""
    if role not in ("throwers", "target") or index not in range(len(CLUBS)):
        return False
    if getattr(self, role) == index:
        return False
    other = "target" if role == "throwers" else "throwers"
    if getattr(self, other) == index:
        setattr(self, other, getattr(self, role))
    setattr(self, role, index)
    self.reset()
    return True
```

Приложение А — select

```
def select(self, kind):
    if kind in range(len(ITEMS)):
        self.selected = kind
```

Приложение А — throw

```
def throw(self):
    """Return whether a new throw was accepted (rapid presses have a cooldown)."""
    if self.cooldown > 1e-9:
        return False
    self.speaker = self.throws % len(FANS)
    x, y = FANS[self.speaker]
    direction = 1 if x < TARGET[0] else -1
    self.projectiles.append(Projectile(
        (x + direction * 29, y - 88),
        (TARGET[0] + self.rng.uniform(-15, 15), TARGET[1] + self.rng.uniform(-10, 10)),
        self.selected,
    ))
    self.poses[self.speaker] = 0.5
    self.bubble = TAUNTS[0] if self.selected == 1 else self.rng.choice(TAUNTS[1:])
    self.bubble = self.bubble.format(target=CLUBS[self.target].name,
throwers=CLUBS[self.throwers].name)
    self.bubble_time = 2.4
    self.throws += 1
    self.cooldown = self.cooldown_seconds
    return True
```

Приложение А — update

```
def update(self, dt):
    """Advance elapsed seconds and return the number of new impacts."""
    if not math.isfinite(dt) or dt < 0:
        raise ValueError("dt must be a finite, nonnegative number")
    self.time += dt
    for name in ("cooldown", "reaction", "bubble_time", "reply_time"):
        setattr(self, name, max(0.0, getattr(self, name) - dt))
    self.poses = [max(0.0, pose - dt) for pose in self.poses]
    for particle in self.particles:
        particle.life -= dt
        particle.x += particle.vx * dt
        particle.y += particle.vy * dt
        particle.vy += 460 * dt
    self.particles = [p for p in self.particles if p.life > 0]
    impacts = 0
    pending = []
    for projectile in self.projectiles:
        projectile.age += dt
        if projectile.progress < 1:
            pending.append(projectile)
            continue
        impacts += 1
        self.hits += 1
        self.reaction = 0.65
        self.last_kind = projectile.kind
        self.reply = self.rng.choice(REPLIES)
        self.reply_time = 1.8
        palette = (
            ((248, 245, 223), (211, 218, 227)),
            ((92, 184, 229), (164, 220, 243)),
            ((246, 196, 81), (131, 202, 183)),
        )[projectile.kind]
        for _ in range(15):
            angle = self.rng.uniform(-math.pi, 0)
            speed = self.rng.uniform(65, 240)
            self.particles.append(Particle(
                *projectile.end, math.cos(angle) * speed, math.sin(angle) * speed,
                self.rng.uniform(0.3, 0.7), self.rng.choice(palette),
            ))
    self.projectiles = pending
    return impacts
```

Приложение Б. Код теста учебной ошибки

```
def test_model_swap_opponent():  
    state = GameState()  
    state.choose_club('target', 0)  
    assert (state.throwers, state.target) == (1, 0)
```

Пример расширенной проверки новой функции:

```
def test_selecting_opponent_swaps_and_clears_inflight_round(self):  
    state = GameState()  
    state.throw()  
    self.assertTrue(state.choose_club('target', 0))  
    self.assertEqual((state.throwers, state.target), (1, 0))  
    self.assertFalse(state.projectiles)  
    self.assertEqual(state.throwers, 0)  
    self.assertFalse(state.choose_club('target', 0))  
    self.assertFalse(state.choose_club('target', 99))  
    self.assertFalse(state.choose_club('unknown', 2))
```

Приложение В. Воспроизведение и состав материалов

Распаковать `experiments_practice2_teams.zip` и открыть терминал в папке с `run_experiments.py`. На Mac с установленным Python 3.15 выполнить:

```
python3.15 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python run_experiments.py
```

На Linux вместо `python3.15` можно использовать `python3`. Измерения отчёта получены на Python 3.12.14; время выполнения и номера строк в трассировках могут зависеть от среды.

Таблица В.1 — Материалы эксперимента

Путь	Содержание
<code>src/</code>	Полный исправный исходный код двух модулей
<code>tests/baseline/</code> , <code>tests/improved/</code>	Базовые и дополненные тесты
<code>run_experiments.py</code>	Правила мутаций, дефекты и автоматический запуск
<code>results/summary.json</code>	Результаты, версии и SHA-256 исходников
<code>results/BUG-*-open/</code> , <code>BUG-*-fixed/</code>	Дефектные копии, журналы до и после исправления
<code>results/baseline-*/</code> , <code>improved-*/</code>	Изменения каждого мутанта и результаты
<code>results/*/htmlcov/</code>	HTML-отчёты о покрытии
<code>results/integration/</code> , <code>menu.png</code>	Журнал smoke-запуска и скриншоты

Контрольная сумма SHA-256 индивидуального исходника:

`a6db5acdbaf2553b53b5d5035776abed44c77725c63a9f4eed6ca2d4d1c816c4`